

Assignment 3 - Stable Fluids

In this assignment you will implement a simple yet stable Eulerian fluid simulation, very similar to the approach described by Stam.

Getting Started

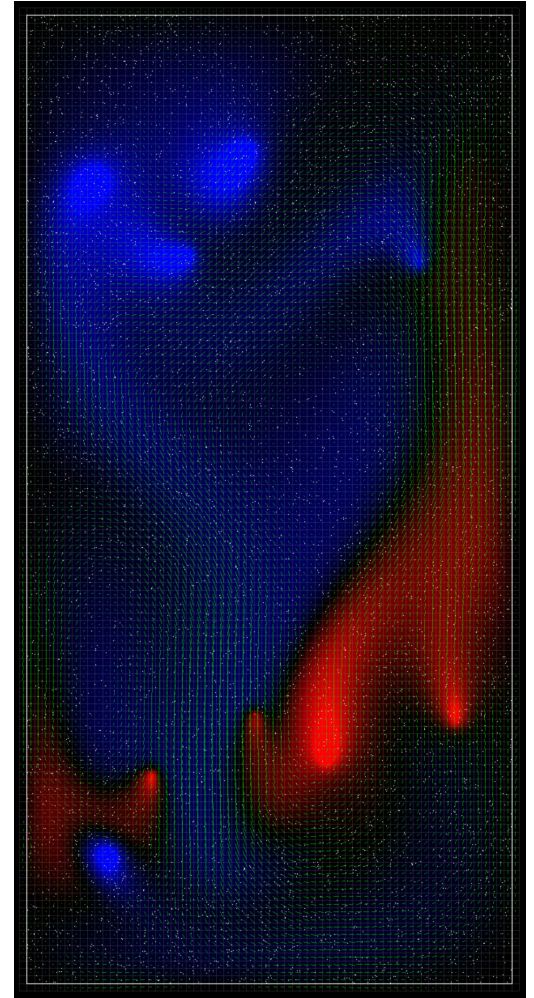
This assignment uses the `moderngl` module for bindings to the OpenGL 3.3+ core functionality, and the corresponding window creation tools from `moderngl-window`. Module `glm` is used to set up an orthographic projection. Otherwise, most of the assignment should be completed with vectorized code using `numpy`.

```
python -m pip install moderngl
python -m pip install moderngl-window
python -m pip install pyglm
python -m pip install numpy
```

Provided Code

The provided code is one file with three main parts. One part of code handles the initialization and drawing in OpenGL. Another part handles the keyboard and mouse input. The final part is the simulation code that you will need to complete.

You will find keyboard and mouse controls useful for testing your code. Dragging while holding the any mouse button will apply forces to your fluid simulation (use the middle button if you don't want to create a temperature source or sink). Clicking the left or right button will create a cold or hot temperature source, respectively. The provided code also implements the following keybindings:



Key	Description
S	Step the simulation
SPACE	Toggle running the simulation
R	Reset the simulation
V	Toggle drawing velocities
LEFT	Decrease velocity line scale
RIGHT	Increase velocity line scale
P	Randomize the particle positions
G	Toggle drawing of grid lines
C	Toggle drawing of colour grid of temperatures
DOWN	Decrease the time step
UP	Increase the time step
COMMA	Decrease GS solver iterations
PERIOD	Increase GS solver iterations
1	Decrease the viscosity
2	Increase the viscosity
3	Decrease the thermal diffusivity
4	Increase the thermal diffusivity
5	Decrease the buoyancy effect
6	Increase the buoyancy effect
DELETE	Removes all temperature sources and sinks
ESCAPE	Quit the program

You will also likely want to manually change the grid size (n_x , n_y) when testing your code.

One of the trickier things to keep track of in the assignment is the grid indexing. Here are some important things to keep in mind:

- All quantities (temperature, velocity) use a grid where the sample is at the cell center, i.e., the provided code does not set up a marker and cell (MAC) grid.
- While n_x and n_y are the number of cells in the domain, there is one cell padding on all boundaries to help you enforce boundary conditions, thus, grids are of size n_x+2 in the horizontal and n_y+2 in the vertical.
- Integers i, j are used for indexing the grid by row and column, i.e., (i, j) corresponds to (y, x) , so you will see grid sizes defined as (n_y+2, n_x+2) .
- The fluid domain is 2 units high, and if it is square (i.e., $n_x == n_y$) then it will also be two units wide. Otherwise it is as wide as necessary for the specified number of grid cells. See that low and high x and y coordinates are stored

in `x1 xh y1 yh` , and the grid domain cell size is stored in `dx` and `dy` , which are equal because the grid cells are squares.

- The window size is in pixels, and there will be a size for each cell in pixels `pix_per_cell` , and this mapping is important for understanding how mouse positions map to the fluid domain
- The function `xy_to_ij` permits mapping of domain points to integer indices along with the cell coordinates of the point, i.e., this function returns the lower left index, along with `a` and `b` sub-cell coordinates that can be used for bi-linear interpolation.
- Overall, you might identify 3 coordinate systems in the code: the window coordinates, the domain coordinates, and then the integer coordinates of the cell grid (along with sub-cell bi-linear coordinates). You should not need to worry about window coordinates unless you modify or add to the mouse interaction code (this is not an objective).

Objectives

Implementing the different objectives in this assignment will often involve adapting code snippets directly from the resources, but you will probably need to understand the code to make it work (i.e., cut and paste may not get you that far).

The code you implement should be vectorized so as to be fast (this is necessary for full marks for different objectives). That is, you will always want to and be able to avoid double for loops to compute different quantities on the grid. See for instance the `xy_to_ij` method which takes points in the domain and maps them to row column indices (`i,j`) and bi-linear interpolation weights (`a,b`). See that this code will naturally work with `np.array` objects, which you can see in `setup_color_mesh` and `draw_color_mesh` where an entire 2D array of positions are converted all at once, and then these values are used to do bi-linear interpolation for the entire 2D array of positions.

Most steps should be trivial to implement and you should be able to test each using the functionality of the provided code.

1. **Advect Particles (1 mark)** Implement the `advect_particles` method by using `xy_to_ij` to compute grid indices (`i,j`) and cell coordinates (`a,b`). Compute the velocity of each particle using bi-linear interpolation of the current grid velocities `curr_v` . You may find it necessary to reshaping of `a` and `b`. Integrate particle positions with forward Euler to compute their new positions after a timestep of `dt` . Use `np.clip` with the bounds of the domain (`x1 xh y1 yh`) to ensure that no particles leave the domain! Test your implementation by carefully dragging the mouse in the domain to create a velocity fields, and reasoning about the motion of the particles when you step the simulation.
2. **Set boundaries (2 mark)** Implement the method `set_boundary` to set the boundary cells (those outside of the domain) for different type of fields. You should set up the boundary according to the provided parameter. When `None` is specified, the outer cells should simply copy those adjacent inside the domain to give a constant interpolation between the cell center and the boundary of the domain (e.g., the boundary used for scalar temperature data). For `vertical` boundary type, the quantity should go to zero at vertical boundaries (e.g., a horizontal velocity must go to zero on the vertical domain boundaries). Similarly the quantity should go to zero at horizontal boundary for 'horizontal' boundary type (e.g., for a vertical velocity field).
3. **Diffusion (1 mark)** Implement the method `diffuse` to do stable diffusion with a backward Euler step. Use the `iterations` member variable as the number of iterations for the Gauss Seidel method. Note that you must also set the appropriate boundaries by calling `set_boundary` after the update.

4. Advect Grid Quantities (1 mark)

Implement the method `advect` to advect quantities in a given scalar field based on a provided velocity field. You also need to set boundaries of the result with specified boundary condition type. You can follow the `advect` code example in GDC03. Optionally, you may consider how you might change the method to trace particles with a higher order integration (see [Stam 1999]). You will test your method in the next step by creating temperature sources and sinks (click left or right mouse button), and observe how these quantities move when you create velocities by dragging the mouse.

5. **Scalar Step (1 mark)** Implement the scalar step. Note that `add_source_temperature` will have already been called by `step`, so you only need to `diffuse` and `advect` the scalar quantities in the temperature field. Use `kappa` as the diffuse rate. The final result must end up in the current temperature field `curr_tp`. Test your method by creating velocity fields with the mouse, and adjusting the diffusion rate with the keyboard controls.

6. **Poisson Solve (2 mark)** Implement the method `project` to "fix" the velocity field so that it is divergence free. Use a Gauss-Seidel solve as per GDC03, and use the same `iterations` parameter you used in the diffusion step. You will also need to set boundaries.

7. **Velocity Step (1 mark)** Implement the velocity step (see GDC03 as reference). Use the `viscosity` parameter for diffusion of the x and y velocities, and be sure to use appropriate boundary conditions. Use the Poisson solve from the previous objective to "fix" the velocity field. The method must also advect the velocity field by itself, and use the `project` method to "fix" the field so that it is (mostly) divergence free. Try adjusting the time step, the viscosity, as well as different grid sizes. Is your solver stable?

8. **Buoyancy Forces (1 mark)** Use the temperature scalar field to compute buoyancy forces to apply to the scalar field in the `apply_temperature_force` method. You will need to compute a reference temperature by computing the mean of the temperature grid. The velocities should be updated (vertical direction only) by the buoyancy coefficient `beta` times the step size time times the temperature delta (comparing the temperature with the reference temperature). You will also need to set boundaries after updating the velocities. For more details, see Foster and Metaxas [1997] Equation 2. Test your implementation by creating hot and cold sources (left and right click), and adjust buoyancy force with the keyboard controls.

9. **MAC Grid (bonus - 3 marks)** That's a lot of bonus marks! That's because there are lots of things that need adjusting if you do the bonus, to the point that you should do your work in a new file called `fluid-mac.py`. The components of the velocity grid will need to be stored separately, and you'll need different indices and coordinates for bi-linear interpolation in these grids. You will need to update the `project` method (see the matlab reference from class). You will also need to adjust (simplify) how you set boundary conditions. You'll also need to update mouse interactions. If you implement the bonus, test your code thoroughly and document your success or remaining issues in your readme file. There is NOT likely to be many part marks available in this bonus, but feel free to make your case if you have a partially working and testable implementation.

References

- J. Stam, [Real-Time Fluid Dynamics for Games](#), GDC 2003.
- J. Stam, [Stable Fluids](#), SIGGRAPH 1999.
- R. Bridson, and M. Müller-Fischer, [Fluid Simulation for Computer Animation](#), SIGGRAPH 2007 Course Notes.
- N. Foster and D. Metaxas, [Modeling the motion of a hot, turbulent gas](#), SIGGRAPH 1997.

Finished?

Great! Submit a zip file using the same structure as the zip you were provided. Be sure to add your name and student number into the comments at the top the file and in the window title. Add any other information you want in a readme.txt file (e.g., request to use your late penalty waiver). Use only zip format, i.e., do not use any other format for your submitted file.

Note that you are encouraged to discuss assignments with your classmates, but not to the point of sharing code and answers. All code and written answers must be your own. Please see the course outline and the fine print in the slides of the first lecture.